Day 1 · Intro to CSS & Selectors

· Introduction to CSS

→ Stands for Cascading Style Sheets. (વેબસાઇટ નો શણગાર - decoration and styling)

→ Controls layout, colors, typography, spacing, and animations across web pages.

→ Syntax: `selector { property: value; }`

> 🎨 *Real-life: HTML is building a bare concrete house; CSS is plaster, colorful paint, and furniture.*

```
h1 { color: blue; font-size: 24px; }
```
:- Makes all <h1> titles blue and 24px

> **Remember:** Always enclose property-value pairs inside curly braces { } and end lines with semicolons (;).

> **Mistake:** Using '=' instead of ':' (e.g. writing color=red; instead of color: red;).

· Core Selectors: Element, Class & ID

→ **Element Selector:** Targets all tags of that type (e.g. `p { ... }`).

→ **Class Selector (.name):** Reusable style across multiple elements
(વર્ગ - ગમે તેટલા તત્વો પર વાપરી શકાય)

→ **ID Selector (#name):** Targets exactly ONE unique element per page
(અનન્ય ઓળખ - એક જ એલિમેન્ટ માટે)

```
.btn { padding: 8px; } | #header { top: 0; }
```
:- Reusable button | Single unique header

> **Remember:** Classes start with a dot (.), IDs start with a hash (#).

> **Mistake:** Reusing the same ID on multiple HTML elements; IDs must be strictly unique!

Day 2 · Comments & Error Handling

· CSS Comments

→ Written using /* comment here */ syntax. (કોડ સમજવા માટે નોંધ)

→ Can span single or multiple lines; completely ignored by the browser.

→ Used to organize sections: Header, Navbar, Footer, Media Queries.

```
/* Main Button Style */ .btn { ... }   :- Explains purpose of the CSS rule
```

Remember: CSS does NOT support // single-line comments! Always use /* ... */.

Mistake: Writing HTML comments <!-- --> or JavaScript // inside CSS files.

· CSS Error Handling

→ CSS fails silently: it never throws fatal crashes or alerts.
(ભૂલ હોય તો તે લાઈન છોડી આગળ વધે)

→ If a browser encounters an unknown property or invalid value, it simply ignores that line and applies the rest.

→ Allows new modern CSS features to be used safely with graceful fallback for older browsers.

```
p { colr: red; font-weight: bold; }   :- 'colr' typo ignored; bold is still applied!
```

Remember: Use browser DevTools (Inspect Element) to spot strikethrough warnings on invalid properties.

Mistake: Assuming silent CSS means code is error-free; always inspect in DevTools!

Day 3 · Colors & Backgrounds

· Color Formats in CSS

→ **Named Colors:** red, blue, coral, darkslateblue (140+ predefined names).

→ **HEX Code:** #ff0000 (Red), #ffffff (White), #000000 (Black).

→ **RGB & RGBA:** `rgb(255, 0, 0)` and `rgba(0, 0, 0, 0.5)` (પારદર્શક રંગ - transparency)

```
color: rgba(0, 100, 255, 0.7);
```
:- Blue with 70% opacity

Remember: The 'A' in RGBA stands for Alpha (opacity from 0.0 transparent to 1.0 solid).

Mistake: Confusing color (text color) with background-color (box fill color).

· CSS Background Properties

→ **background-color:** Fills background with a solid color. (પાછળ નો રંગ)

→ **background-image:** `url('bg.jpg')` loads an image. (પાછળ નો ફોટો)

→ **background-repeat:** `no-repeat` prevents tiling.

→ **background-size:** `cover` scales image to fill the entire container proportionally.

```
background: url('bg.jpg') no-repeat center/cover;
```
:- Perfect full-cover background image

Remember: Always provide a fallback background-color in case the background image fails to load!

Mistake: Forgetting no-repeat; small background images will tile across the whole page by default.

"Colors, like features, follow the changes of the emotions"

Day 4 · Borders & Rounded Corners

· CSS Borders

→ Surrounds elements with a line. Has width, style, and color. (કિનારી / બોર્ડર)

→ **Styles:** solid, dashed, dotted, double, groove, none.

→ **Shorthand:** `border: 2px solid #3b82f6;`

`border-bottom: 3px dashed red;` :- Dashed red underline below the box

Remember: If border-style is not specified, border defaults to 'none' and will NOT show!

Mistake: Writing border: 2px red; without specifying 'solid' or 'dashed' (border won't appear!).

· Rounded Corners: border-radius

→ Curves the sharp 90-degree corners of any box or button. (ખૂણા ગોળ કરવા)

→ Small curves: `border-radius: 6px;`

→ Pill buttons: `border-radius: 9999px;`

→ Perfect circle: `width: 100px; height: 100px; border-radius: 50%;`

`border-radius: 50%; width: 80px; height: 80px;` :- Turns square avatar into a perfect circle

Remember: For border-radius: 50% to produce a circle, width and height MUST be equal!

Mistake: Setting border-radius: 50% on a rectangular image; it becomes an oval, not a circle.

"Good boundaries create clarity and pleasant aesthetics"

Day 5 · Margin, Padding & Box Model

· Margin vs Padding

→ **Margin:** Space OUTSIDE the border (pushes neighbor elements away).
(બોર્ડર ની બહાર ની ખાલી જગ્યા)

→ **Padding:** Space INSIDE the border (cushions text inside the box).
(બોર્ડર ની અંદર ની ખાલી જગ્યા)

→ **Shorthand (Clockwise):** `margin: Top Right Bottom Left;` (TRBL)

`padding: 10px 20px;` :- 10px Top/Bottom, 20px Left/Right

Remember: margin: 0 auto; centers a block element horizontally within its parent container!

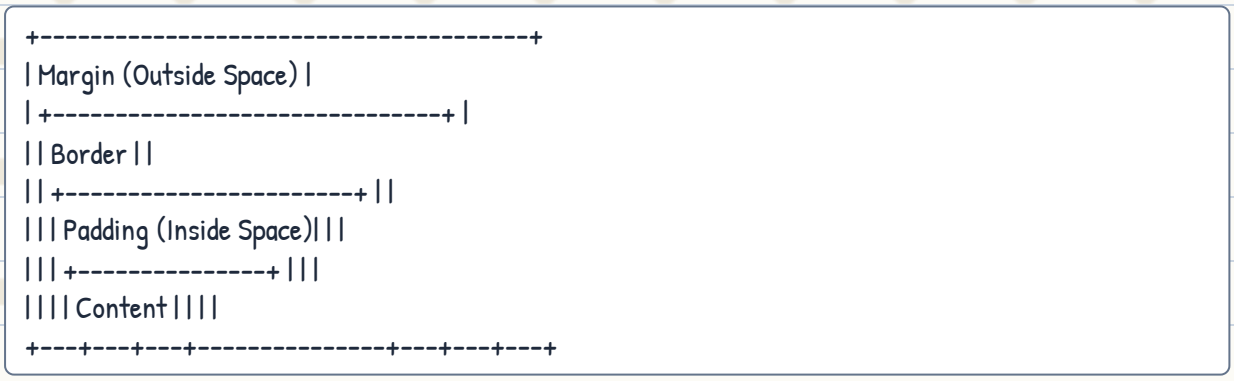
Mistake: Using margin when you want to increase clickable area of a button; use padding instead.

· The CSS Box Model & box-sizing

→ Every element is a rectangular box made of: Content → Padding → Border → Margin.

→ **Standard box-sizing: content-box** adds padding & border on TOP of width (causes horizontal overflow!).

→ **border-box:** Keeps total width fixed; padding and border stay INSIDE the specified width.
(સાઈઝ બહાર ન નીકળે)



Remember: Always put * { box-sizing: border-box; } at the top of every CSS project!

Mistake: Leaving default content-box; adding 20px padding to a 100% wide box creates scrollbars.

Day 6 · Width, Height & Max-Width

· Width & Height Properties

→ Sets the width and height of block-level and inline-block elements. (પહોળાઈ અને ઊંચાઈ)

→ Can be set in fixed units (px) or relative units (%, vh, vw).

→ auto lets browser calculate based on content size.

```
div { width: 50%; height: 200px; }   :- Half width of screen, 200px tall
```

Remember: Inline elements (like or <a>) IGNORE width and height properties!

Mistake: Setting hardcoded height in pixels on text cards; content will overflow when text grows.

· max-width for Responsive Design

→ **max-width:** Restricts element from growing beyond a limit, but allows it to shrink on smaller screens. (મહત્તમ પહોળાઈ - સ્ક્રીન નાની થાય ત્યારે સંકોચાય)

→ **Responsive images rule:** `img { max-width: 100%; height: auto; }` prevents image from spilling off screen.

```
.container { max-width: 1200px; width: 100%; margin: 0 auto; }   :- Responsive centered website container
```

Remember: Prefer max-width over fixed width for main layout wrappers and images!

Mistake: Using width: 1200px on mobile; screen will force a clumsy horizontal scrollbar.

Day 7 · Outline & Text Styling

· CSS Outline vs Border

→ **Outline:** Drawn outside the border edge; does NOT take up any layout space. (બહાર ની વધારાની લાઈન)

→ Does not affect document flow or push adjacent elements.

→ Most commonly used for accessibility keyboard focus indicators (`:focus-visible`).

```
input:focus { outline: 2px solid blue; }   :- Blue highlight box when user clicks input
```

Remember: Never set outline: none without providing an alternative focus indicator for keyboard users!

Mistake: Using outline: none on forms; blind and keyboard users will lose track of where they are.

· Text Formatting Properties

→ **color:** Text color | **text-align:** left, center, right, justify. (લખાણ ની ગોઠવણી)

→ **text-decoration:** none (removes link underline), underline, line-through.

→ **text-transform:** uppercase, lowercase, capitalize. (મોટા કે નાના અક્ષરો)

→ **line-height:** Distance between lines of text (ideal: 1.5 to 1.6 for readability).

```
a { text-decoration: none; text-transform: uppercase; }   :- Clean uppercase links without underline
```

Remember: Always use line-height: 1.5+ on body text for comfortable reading experience.

Mistake: Using text-align: justify on websites; it creates awkward 'rivers' of empty white gaps.

Day 8 · Fonts & Icons

· Typography & Font Properties

→ **font-family:** Specifies font face with fallbacks: `'Inter', Arial, sans-serif;`
(ફોન્ટ ની શૈલી)

→ **font-size:** Text size (e.g. `16px`, `1.25rem`).

→ **font-weight:** Thickness from thin (100), regular (400), semi-bold (600), to bold (700).

→ **Google Fonts:** Import via `@import url('...');` in CSS or <link> in HTML.

```
font-family: 'Patrick Hand', cursive, sans-serif;   :- Applies handwriting font with fallbacks
```

Remember: Always include a generic fallback (sans-serif, serif, or monospace) at the end of font-family!

Mistake: Using 5 different font families on one webpage; stick to 1 or 2 fonts maximum.

· Web Icons (FontAwesome & SVG)

→ Icons can be added using icon font libraries (Font Awesome) or inline SVGs.
(આઇકોન્સ - પ્રતીકો)

→ Icon fonts can be styled just like regular text using `font-size` and `color`!

→ SVGs provide sharp crisp rendering on all retina high-res screens.

```
.icon { font-size: 24px; color: #f59e0b; }   :- Sizes and colors icon like normal text
```

Remember: SVG icons are resolution-independent and will never get blurry on zoom!

Mistake: Using low-res PNG files for icons; they look pixelated on mobile phones.

Day 9 · Links & Lists Styling

· Styling Hyperlink States (LVHA Rule)

→ Links have 4 interactive pseudo-classes: (લિંક ની અલગ અલગ સ્થિતિ)

→ **:link** (normal unvisited) → **:visited** (already visited page)

→ **:hover** (mouse over) → **:active** (moment link is clicked).

```
a:hover { color: orange; text-decoration: underline; }
```
:- Highlights when cursor hovers

Remember: Follow the LVHA order strictly in CSS: :link → :visited → :hover → :active!

Mistake: Writing :hover before :link in CSS; the hover effect will be overridden and fail!

· Styling HTML Lists

→ **list-style-type:** none, disc, circle, square, decimal, lower-alpha. (લિસ્ટ ના ટપકાં બદલવા)

→ **list-style: none;** Removes default black dots and numbers.

→ To reset a list for navbars: `list-style: none; padding: 0; margin: 0;`

```
ul.nav { list-style: none; display: flex; gap: 15px; }
```
:- Turns list into horizontal navigation bar

Remember: Browser adds default padding-left (40px) to <ul>; remember to zero it out when resetting!

Mistake: Setting list-style: none and wondering why items are indented (forgot to set padding: 0;).

"Navigation should be a guide, not a puzzle"

Day 10 · Tables & Display Property

· CSS Table Styling

→ **border-collapse: collapse;** Merges double border lines into a single clean line.
(ડબલ લાઈન ભેગી કરી સિંગલ લાઈન બનાવવી)

→ Zebra striping: `tr:nth-child(even) { background: #f1f5f9; }`
(એક પછી એક પટ્ટા વાળો રંગ)

→ Hover effect: `tr:hover { background: #e2e8f0; }`

```
table { width: 100%; border-collapse: collapse; }   :- Clean, full-width single-bordered table
```

> **Remember:** Without border-collapse: collapse, HTML tables show an ugly double border around cells!

> **Mistake:** Forgetting to add padding to <th> and <td>; text will touch the border lines.

· The CSS display Property

→ Controls how an element is rendered in page flow. (એલિમેન્ટ નો પ્રદર્શન પ્રકાર)

→ **block:** Full width, starts new line (div, p).

→ **inline:** Flow within sentence, ignores width/height (span, a).

→ **inline-block:** Flows like inline, BUT respects width, height, and margins!

→ **none:** Completely hides element and removes its space from document.

```
.btn { display: inline-block; padding: 10px 20px; }   :- Allows padding & width on link buttons
```

> **Remember:** display: none hides element completely; visibility: hidden hides it but reserves empty space!

> **Mistake:** Trying to apply width & top/bottom margin to inline elements without setting inline-block.

Day 11 · CSS Positions

· Static, Relative & Absolute Positioning

→ **position: static:** Default normal page flow; top/left/right/bottom have no effect. (સામાન્ય ગોઠવણી)

→ **position: relative:** Shifted relative to its normal position without disturbing others. (પોતાની જગ્યા થી ખસવું)

→ **position: absolute:** Removed from normal flow; positioned relative to nearest positioned parent (with relative/absolute). (પેરેન્ટ ના આધારે ગમે ત્યાં મૂકવું)

```
.parent { position: relative; } .badge { position: absolute; top: -5px; right: -5px; }
```
:- Notification badge on top corner

Remember: Always give position: relative to parent before putting position: absolute on children!

Mistake: Setting position: absolute without a relative parent; the child jumps to the top of <body>!

· Fixed & Sticky Positioning

→ **position: fixed:** Locked to the browser viewport; stays in exact place even when scrolling. (સ્ક્રીન પર ચોંટી રહેવું - e.g. floating chat/navbar)

→ **position: sticky:** Scrolls normally until reaching offset (e.g. `top: 0`), then sticks like fixed! (સ્ક્રોલ થતા ટોપ પર ચોંટી જવું)

```
nav { position: sticky; top: 0; z-index: 100; }
```
:- Header that sticks to top when user scrolls down

Remember: position: sticky requires at least one direction offset (top, bottom, left, or right) to work!

Mistake: Using position: sticky inside an ancestor that has overflow: hidden (sticky will NOT work!).

Day 12 · Z-Index & Overflow

· Layer Stacking with z-index

→ Controls vertical stacking order of overlapping elements (3rd dimension Z-axis).
(કયું એલિમેન્ટ ઉપર દેખાશે - stacking order)

→ Higher number appears on top of lower numbers (e.g. `z-index: 10` beats `z-index: 1`).

→ Can be negative (`z-index: -1` places background behind content).

```
.modal { position: fixed; z-index: 1000; }
```
:- Pops up above all regular website elements

Remember: z-index ONLY works on elements with a position property (relative, absolute, fixed, sticky)!

Mistake: Adding z-index: 9999 to position: static element and wondering why it doesn't come on top.

· Handling Spills with overflow

→ Controls what happens when content exceeds its box container's width/height.
(લખાણ બોક્સ ની બહાર નીકળે ત્યારે)

→ **visible:** Default; spills outside the box boundary.

→ **hidden:** Clips the overflow; extra content becomes invisible.

→ **scroll:** Always shows scrollbars (even if content fits).

→ **auto:** Smart; adds scrollbar ONLY when content actually overflows.

```
.card-body { height: 200px; overflow-y: auto; }
```
:- Clean vertical scroll when text gets long

Remember: Use overflow: auto instead of overflow: scroll to avoid empty disabled scrollbars!

Mistake: Setting overflow: hidden on dropdown menus; the dropdown list will be cut off and hidden!

Day 13 · Floats & Alignment

· Float, Clear & Clearfix

→ **float: left / right:** Pushes element to side and lets text wrap around it (magazine style).
(એલિમેન્ટ ને ડાબે કે જમણે તરતું મૂકવું)

→ **clear: both:** Prevents following elements from wrapping around floated items.

→ **Clearfix Hack:** When parent collapses because all children are floated, add `overflow: auto;` or `.clearfix::after`.

```
img { float: left; margin-right: 15px; }
```
:- Article text wraps neatly around the image

> **Remember:** Use Flexbox and Grid for modern layouts; reserve float strictly for text-wrapping images!

> **Mistake:** Floating child elements without clearing; parent height collapses to 0px!

· Centering & Alignment Methods

→ **Center inline text / images:** `text-align: center;` on parent container.
(લખાણ વચ્ચે લાવવું)

→ **Center block box:** `margin: 0 auto; width: 600px;` on the block itself.
(બોક્સ ને વચ્ચે લાવવું)

→ **Center both vertically & horizontally:** `display: flex; justify-content: center; align-items: center;`

```
.box { width: 400px; margin: 0 auto; }
```
:- Centers a 400px wide card on the screen

> **Remember:** margin: 0 auto ONLY works if the element has a specified width and is display: block!

> **Mistake:** Putting text-align: center on a div expecting the div box itself to center (it only centers text inside!).

Day 14 · CSS Combinators

· Descendant & Child Combinators

→ Combinators explain relationship between multiple selectors. (સિલેક્ટર્સ વચ્ચે નો સંબંધ)

→ **Descendant Selector (space):** Targets ALL matching children, grandchildren, and deeper. (અંદર ના તમામ વારસદારો)

→ **Child Selector (>):** Targets ONLY direct immediate children (1 level deep). (ફક્ત સીધો પુત્ર - direct child)

`div p { ... }` vs `div > p { ... }` :- All paragraphs inside div vs only direct child paragraphs

Remember: Use child selector (>) when you want to avoid accidentally styling deeply nested elements!

Mistake: Using space selector when styling navbar dropdowns; sub-dropdown items inherit unwanted styles.

· Sibling Combinators (+ and ~)

→ **Adjacent Sibling (+):** Targets the element immediately following another element. (તરત પછી નું પડોશી એલિમેન્ટ)

→ **General Sibling (~):** Targets all siblings that follow an element at the same level. (પછી ના તમામ પડોશી એલિમેન્ટ્સ)

```
h2 + p { font-size: 18px; font-weight: bold; }
```
:- Styles only the lead paragraph directly following h2

Remember: The adjacent sibling (+) is perfect for styling the first paragraph after a heading!

Mistake: Thinking '+' targets preceding elements; combinators only target elements that appear AFTER.

Day 15 · Pseudo-classes & Elements

· CSS Pseudo-Classes (:)

→ Defines a special STATE of an element (single colon `:`). (એલિમેન્ટ ની ખાસ પરિસ્થિતિ)

→ **:hover:** Mouse cursor is over the element.

→ **:focus:** Form input is actively selected.

→ **:first-child & :last-child:** First / last child of parent.

→ **:nth-child(2n):** Every even child (used for zebra striping tables).

```
button:hover { transform: translateY(-2px); }   :- Subtle lift animation on mouse hover
```

Remember: :nth-child() is 1-indexed in CSS (first element is 1, not 0)!

Mistake: Using :nth-child(0); it matches nothing because CSS child counting starts at 1.

· CSS Pseudo-Elements (::)

→ Styles a specific PART of an element or creates virtual elements (double colon `::`). (વર્ચ્યુઅલ એલિમેન્ટ બનાવવું)

→ **::before & ::after:** Injects decorative content before or after element without HTML tags!

→ **::first-letter:** Drop-cap initial letter like in storybooks.

→ **::selection:** Styles highlighted text selection color.

```
.star::after { content: ' ★'; color: gold; }   :- Adds gold star after text via CSS
```

Remember: ::before and ::after MUST always have `content: '';` (even if empty) to render!

Mistake: Omitting the content property on ::before/::after; the pseudo-element will not display.

"The unseen details create the magic of modern interfaces"

Day 16 · Navbars & Dropdowns

· Building Clean Navbars

→ Navbars are built using semantic `<nav>` containing a `<ul>`. (નેવિગેશન મેનુ બાર)

→ Remove bullet dots: `list-style: none;`

→ Horizontal alignment: `display: flex; gap: 20px; align-items: center;`

→ Give links generous padding (`padding: 8px 16px;`) for comfortable tap targets on touchscreens.

`nav ul { display: flex; list-style: none; gap: 20px; }` :- Clean modern horizontal menu

Remember: Make the entire link clickable with padding, rather than making user click tiny text letters!

Mistake: Applying padding to <li> instead of <a>; only the letters will be clickable.

· Pure CSS Dropdown Menus

→ Parent list item gets `position: relative;`. (ડ્રોપડાઉન સબ-મેનુ)

→ Sub-menu gets `position: absolute; display: none; top: 100%; left: 0;`

→ On hover: `.nav-item:hover .sub-menu { display: block; }` reveals menu!

```
[ Products v ] ← parent (position: relative)
  |-- [ Laptops ]
  |-- [ Mobiles ] ← sub-menu (position: absolute; display: none)
```

Remember: Add a subtle box-shadow (0 4px 12px rgba(0,0,0,0.15)) to dropdowns for depth!

Mistake: Leaving a gap between parent button and dropdown list; mouse hover is lost when moving across gap.

"Good navigation leads visitors where they want to go with zero friction"

Day 17 • Opacity & Attribute Selectors

• Opacity vs RGBA Transparency

→ **opacity:** Sets transparency of the ENTIRE element and all its children (0.0 to 1.0).
(આખું એલિમેન્ટ અને અંદર નું લખાણ ઝાંખું કરવું)

→ **rgba():** Sets transparency of background ONLY; text inside stays 100% solid and crisp!
(ફક્ત બેકગ્રાઉન્ડ પારદર્શક રાખવું)

```
background: rgba(0, 0, 0, 0.6); color: #fff; :- Semi-transparent dark card with crisp white text
```

Remember: Use rgba() or hex alpha for translucent backgrounds so text readability isn't ruined!

Mistake: Using opacity: 0.5 on a modal dialog box; text inside becomes washed out and hard to read.

• CSS Attribute Selectors

→ Targets elements based on presence or value of HTML attributes.
(એટ્રિબ્યુટ ના આધારે પસંદગી)

→ **[target]:** Elements having target attribute.

→ **[type="submit"]:** Exact attribute match.

→ **[href^="https"]:** Starts with 'https' (secure links).

→ **[src$=".pdf"]:** Ends with '.pdf' (PDF downloads).

```
input[type="email"] { border-color: #3b82f6; }  :- Styles only email input fields
```

Remember: Attribute selectors allow styling form inputs without adding extra class names!

Mistake: Typing attribute selectors without brackets (e.g. type="text" instead of [type="text"]).

"Precision in selection leads to elegance in design"

Day 18 · Form Styling & CSS Counters

· Professional Form Styling

→ Reset ugly browser defaults: `border: 1.5px solid #cbd5e1; border-radius: 6px; padding: 10px 14px;`

→ Smooth focus transition: `input:focus { border-color: #3b82f6; box-shadow: 0 0 0 3px rgba(59,130,246,0.2); }`

(ઈનપુટ સિલેક્ટ થાય ત્યારે સુંદર ગ્લો)

→ Style placeholders: `::placeholder { color: #94a3b8; font-style: italic; }`

`input:focus { outline: none; border-color: #2563eb; }` :- Professional focused glow effect

Remember: Always provide a visual focus state on inputs so users know where their cursor is!

Mistake: Removing browser default outline with outline: none and providing no replacement border focus.

· CSS Counters (Automatic Numbering)

→ CSS variables that automatically count items in a page without manual numbering.

(આપોઆપ ક્રમ આપવો - 1, 2, 3)

→ **counter-reset: name;** Initializes counter to 0 (placed on parent container).

→ **counter-increment: name;** Increments counter by 1 on each item.

→ **content: counter(name);** Displays number inside ::before pseudo-element!

`ol { counter-reset: c; } li::before { counter-increment: c; content: counter(c) '. '; }` :- Fully customized list numbering

Remember: CSS counters are ideal for multi-step wizards (Step 1, Step 2, Step 3)!

Mistake: Placing counter-reset on the list items themselves; the counter will reset to 0 every time!

"Small automated touches make code graceful"

Day 19 · Units & Specificity

· CSS Units: Absolute vs Relative

→ **px (Pixels):** Absolute fixed unit; does not scale with user browser zoom settings. (ફિક્સ પિક્સેલ્સ)

→ **rem (Root EM):** Relative to root `<html>` font size (1rem = 16px default). Best for accessible typography! (રૂટ ફોન્ટ સાઈઝ ના આધારે - શ્રેષ્ઠ પદ્ધતિ)

→ **em:** Relative to parent element font size (can compound unintentionally).

→ **vw / vh:** 1% of Viewport Width / Viewport Height (100vh = full screen height). (સ્ક્રીન ની ટકાવારી)

```
h1 { font-size: 2rem; } /* 32px */ body { min-height: 100vh; }
```
:- Scalable heading & full-height page

Remember: Use rem for fonts and spacing; use % or vh/vw for responsive layouts!

Mistake: Nesting em units deeply; font size multiplies and becomes unexpectedly huge or tiny.

· CSS Specificity Hierarchy

→ Determines which rule wins when multiple CSS rules target the same element. (કયો નિયમ જીતશે - priority order)

→ **1. !important:** Overrides everything (use sparingly!).

→ **2. Inline styles:** `style="..."` (Weight: 1000)

→ **3. ID selector:** `#header` (Weight: 100)

→ **4. Classes / Attributes:** `.btn, [type]` (Weight: 10)

→ **5. Elements:** `p, div` (Weight: 1)

```
!important > Inline (1000) > #id (100) > .class (10) > tag (1)
```

Remember: Higher specificity wins; if specificity is equal, the rule written LATER in the file wins!

Mistake: Overusing !important to patch bugs; it destroys maintainability and cascades into CSS wars.

Day 20 · Performance & Gradients

· CSS Performance Optimization

→ **Minimize Reflows & Repaints:** Animating `transform` and `opacity` uses GPU and is 60fps fast. (ઝડપી સ્પીડ - GPU એનિમેશન)

→ Avoid animating layout properties: `top, left, width, height, margin` cause slow CPU recalculations.

→ Keep selectors shallow (avoid `div body #main ul li a span`).

→ Minify CSS files in production to eliminate whitespace and comments.

```
/* Fast */ transform: translateX(100px); /* Slow */ left: 100px;
```
:- Hardware accelerated smooth movement

Remember: Always animate transform and opacity for buttery-smooth 60fps animations!

Mistake: Animating margin-left or top; mobile phones will stutter and lag.

· CSS Gradients (Linear & Radial)

→ Smooth transitions between two or more colors without downloading heavy images. (રંગો નું મિશ્રણ - ઢાળવાળા રંગો)

→ **linear-gradient(direction, color1, color2):** `to right, #4facfe, #00f2fe;`

→ **radial-gradient(circle, color1, color2):** Radiates outward from center.

```
background: linear-gradient(135deg, #667eea, #764ba2);
```
:- Elegant modern purple gradient

Remember: Gradients are treated as background-image, not background-color!

Mistake: Writing background-color: linear-gradient(...); (it will be ignored; use background:).

Day 21 · Shadows & Text Effects

· box-shadow & text-shadow

→ **box-shadow:** Adds elevation and 3D depth to cards and buttons.
(કાર્ડ નો પડછાયો - 3D elevation)

→ Syntax: `box-shadow: X Y Blur Spread Color;`

→ **text-shadow:** Adds shadow behind letters for contrast on photo backgrounds.

`box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);` :- Soft modern card drop-shadow

Remember: Use multiple subtle shadows with soft blur for realistic, premium lighting!

Mistake: Using harsh black shadows (like box-shadow: 5px 5px 0 #000); looks 90s and dated.

· Text Truncation & Ellipsis (...)

→ When text is too long for a single line, cut it cleanly with three dots (...).
(લાંબુ લખાણ કાપીને ... બતાવવું)

→ **Must combine 3 properties:**

→ 1. `white-space: nowrap;` (prevents wrapping)

→ 2. `overflow: hidden;` (clips overflow)

→ 3. `text-overflow: ellipsis;` (appends ...)

`white-space: nowrap; overflow: hidden; text-overflow: ellipsis;` :- Shows: 'Very long product title...'

Remember: All 3 properties are required together; if you miss one, ellipsis will NOT appear!

Mistake: Expecting text-overflow: ellipsis to work without overflow: hidden.

"Light and shadow give shape and dimension to flat screens"

Day 22 · 2D & 3D Transforms

· 2D Transforms: translate, rotate, scale

→ Modifies element shape, size, and position without disrupting document flow. (આકાર અને સ્થિતિ બદલવી)

→ **translate(X, Y):** Moves element (e.g. `translateY(-5px)` lifts up).

→ **rotate(deg):** Spins element (e.g. `rotate(45deg)`).

→ **scale(N):** Enlarges or shrinks (e.g. `scale(1.05)` zooms 5%).

```
button:hover { transform: scale(1.05); }
```
:- Micro-zoom button effect on hover

Remember: Combine multiple transforms in one rule: transform: translate(-50%, -50%) rotate(45deg);

Mistake: Writing two separate transform lines; the second line completely overrides the first!

· 3D Transforms & Perspective

→ Simulates real-world 3D depth along the Z-axis. (3D પરિમાણ - ઊંડાણ)

→ **perspective: 1000px;** Defines how far the viewer is from the 3D plane (placed on parent).

→ **rotateX(), rotateY():** Flips card along horizontal and vertical axes.

→ **transform-style: preserve-3d;** Keeps children in 3D space (for 3D card flips).

```
.card:hover { transform: rotateY(180deg); }
```
:- 3D card flip showing backside

Remember: Always add perspective to parent container to enable realistic 3D foreshortening!

Mistake: Forgetting perspective; rotateY will look like a flat shrinking box rather than 3D rotation.

"Transformations turn static graphics into lively experiences"

Day 23 · Transitions & Animations

· CSS Transitions (Smooth State Changes)

→ Smoothly animates property changes over time (like on :hover).
(સરળ ફેરફાર - smooth change)

→ **Shorthand:** `transition: property duration timing-function delay;`

→ **ease-in-out:** Starts slow, speeds up in middle, ends slow for natural feel.

```
transition: all 0.3s ease-in-out;
```
:- Smooth 0.3 second hover animation

Remember: Put transition on the BASE class (e.g. .btn), not on :hover, so it animates both ways!

Mistake: Putting transition on :hover only; the hover-in is smooth, but hover-out snaps abruptly.

· Keyframe Animations (@keyframes)

→ Creates continuous, multi-step animations without requiring user interaction.
(આપોઆપ ચાલતી એનિમેશન)

→ **@keyframes name:** Defines checkpoints (0%, 50%, 100%).

→ **animation:** `name duration timing iteration-count;`

```
@keyframes spin { 100% { transform: rotate(360deg); } }
.loader { animation: spin 1s linear infinite; }
```
:- Infinite spinning loading spinner

Remember: Use iteration-count: infinite for continuous animations like loading spinners!

Mistake: Forgetting to bind animation-name to the element class after defining @keyframes.

"Animation is not the art of drawings that move, but the art of movements that are drawn"

Day 24 · Tooltips & Image Styling

· Pure CSS Tooltips

→ Small popup bubble explaining an icon or button when hovered.
(માઉસ લઈ જતાં નાની મદદગાર માહિતી બતાવવી)

→ Built with `data-tooltip` attribute and `::after` pseudo-element.

→ Positioned with `position: absolute; bottom: 125%; left: 50%; transform: translateX(-50%);`

```
.tip:hover::after { content: attr(data-tooltip); opacity: 1; }   :- Fades in tooltip text on hover
```

Remember: Use attr(data-tooltip) in CSS content so HTML controls the tooltip text dynamically!

Mistake: Setting z-index too low on tooltip; it ends up hidden behind neighboring cards.

· Image Styling: object-fit

→ Controls how an image or video fits inside a container with fixed width/height.
(ફોટો ખેંચાયા વગર બોક્સમાં ફીટ કરવો)

→ **fill:** Default; stretches and distorts image aspect ratio.

→ **cover:** Crops edges to fill the container without distorting proportions! (Best for avatars & cards) (પ્રમાણસર કવર કરવું)

→ **contain:** Letterboxes image so the whole photo is visible without cropping.

```
img { width: 100%; height: 250px; object-fit: cover; }   :- Perfectly cropped card image without distortion
```

Remember: Always pair fixed height with object-fit: cover on image cards!

Mistake: Squishing images by forcing width & height without object-fit; people look fat or skinny.

"Helpful hints make complex interfaces intuitive"

Day 25 · Filters, Clip-path & Multi-columns

· CSS Image Filters & clip-path

→ **filter:** Visual effects: `blur(5px)`, `grayscale(100%)`, `brightness(1.2)`, `drop-shadow()`. (ઈમેજ ઈફેક્ટ્સ - બ્લેક એન્ડ વ્હાઈટ, બ્લર)

→ **clip-path:** Cuts elements into geometric custom shapes (polygons, circles, triangles). (ખાસ આકારમાં કાપવું - polygon/circle)

→ Example: Slanted hero sections via `clip-path: polygon(0 0, 100% 0, 100% 85%, 0 100%);`

```
img:hover { filter: grayscale(0%); } img { filter: grayscale(100%); }
```
:- Black & white photo turns vibrant on hover

Remember: drop-shadow() filter conforms to transparent PNG edges, unlike standard box-shadow!

Mistake: Using box-shadow on a transparent PNG; it casts a shadow around the rectangular box, not the icon.

· CSS Multi-column Layout

→ Splits text into newspaper-style multi-column layout. (અખબાર જેવા સ્તંભો - newspaper columns)

→ **column-count: 3;** Divides text into 3 equal columns.

→ **column-gap: 30px;** Space between adjacent columns.

→ **column-rule: 1px solid #ccc;** Draws vertical dividing line between columns.

```
.article { column-count: 2; column-gap: 24px; }
```
:- 2-column newspaper layout for article text

Remember: Use column-width instead of column-count for responsive auto-adjusting column counts!

Mistake: Using hardcoded column-count: 3 on mobile screens; text columns become unreadably narrow.

"Artistic filters and geometric shapes elevate standard layouts into editorial art"

Day 26 · CSS Variables (Custom Properties)

· Declaring & Using CSS Variables

→ Allows defining reusable design values (colors, spacing, font sizes) in one place.
(વેરીએબલ - એક જગ્યાએ ફેરફાર કરવાથી આખી સાઈટ બદલાય)

→ Declared with two dashes (`--name`) inside `:root` for global scope.

→ Accessed using `var(--name, fallback)`.

```
:root { --primary: #3b82f6; }
button { background: var(--primary); }
```
:- Uses centralized primary blue variable

Remember: Variables cascade! You can override --primary inside a specific card for thematic coloring.

Mistake: Forgetting the two dashes '--' prefix when defining CSS custom properties.

· Dark Mode with CSS Variables

→ Variables make instant Dark Mode implementation effortless. (ડાર્ક મોડ - રાત્રિ મોડ)

→ Define base theme on `:root`, then redefine color variables inside `[data-theme='dark']` or `@media (prefers-color-scheme: dark)`.

```
:root {
  --bg: #ffffff; --text: #1e293b;
}
[data-theme='dark'] {
  --bg: #0f172a; --text: #f8fafc;
}
```

Remember: All elements using var(--bg) and var(--text) will swap themes instantaneously!

Mistake: Hardcoding hex codes on child elements; they will refuse to change in dark mode.

"Don't repeat yourself. Centralize your design tokens"

Day 27 · Flexbox Layout

· Flex Container: Direction & Spacing

→ **display: flex;** Activates 1-dimensional layout along main and cross axes.
(એક પરિમાણીય લવચીક ગોઠવણી)

→ **flex-direction:** row (horizontal default) or column (vertical stack). (આડી કે ઊભી દિશા)

→ **justify-content:** Aligns items along main axis (flex-start, center, flex-end, space-between, space-around).

→ **align-items:** Aligns items along cross axis (stretch, center, flex-start).

→ **gap:** Simple space between items (e.g. `gap: 16px;`).

```
display: flex; justify-content: space-between;
align-items: center;
```
:- Items pushed to opposite ends and centered vertically

Remember: justify-content aligns along main axis; align-items aligns along perpendicular cross axis!

Mistake: Using space-between and then adding margins on items; spacing becomes uneven.

· Flex Items: grow, shrink, wrap

→ **flex-wrap: wrap;** Allows items to wrap onto the next line if space runs out.
(જગ્યા ન હોય તો બીજી લાઈનમાં જવું)

→ **flex-grow: 1;** Tells item to expand and take up all remaining available space.

→ **flex-shrink: 0;** Prevents item from being squished when space is tight.

```
.input { flex: 1; } .btn { flex-shrink: 0; }
```
:- Search input stretches; search button stays fixed size

Remember: flex: 1 is shorthand for flex-grow: 1, flex-shrink: 1, flex-basis: 0%!

Mistake: Items overflowing off screen because flex-wrap: wrap was omitted on responsive rows.

"Flexibility is the greatest strength in a world of varying screen sizes"

Day 28 · Media Queries & CSS Grid

· Media Queries for Responsive Design

→ Applies CSS rules conditionally based on screen width, height, or orientation.
(મોબાઈલ અને ડેસ્કટોપ માટે અલગ નિયમ)

→ **Mobile First approach:** Write default styles for mobile, then add `@media (min-width: 768px)` for tablets and `(min-width: 1024px)` for desktop laptops.

```
@media (max-width: 768px) {
  .nav { flex-direction: column; }
}
```
:- Stacks navbar vertically on mobile phones

Remember: Always include <meta name="viewport" content="width=device-width, initial-scale=1.0"> in HTML!

Mistake: Using fixed pixel widths inside media query blocks; use percentages or fr units.

· CSS Grid (2D Grid Layout)

→ **display: grid;** Powerful 2-dimensional system handling rows AND columns simultaneously.
(દ્વિ-પરિમાણીય ગ્રીડ - રો અને કોલમ બંને)

→ **grid-template-columns:** `repeat(3, 1fr);` (3 equal flexible fractional columns).

→ **grid-column: 1 / 3;** Item spans across 2 columns. (બે કોલમ રોકી લેવી)

→ **Auto-responsive magic:** `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));` (no media queries needed!)

```
display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 16px;
```
:- Responsive auto-wrapping grid cards

Remember: Use Flexbox for 1D navigation bars and rows; use CSS Grid for 2D page layouts and card galleries!

Mistake: Trying to build complex 2D dashboard grids using nested flexboxes; CSS Grid does it with 3 lines.

"Responsive design is not about designing for devices, but designing for people"